



Øving 3: Splitt og hersk

Kollapser kontekst

←

→

Du må oppnå minst 11 av totalt 28 mulige poeng for å denne øvingen godkjent. Hver flervalgsoppgave er verdt 1 poeng, mens hver kodeoppgave er verdt 3 poeng. For informasjon om øvingsopplegget, se nettsidene. Trenger du hjelp, så ta kontakt.

Løsningsforslaget publiseres på Canvas etter innleveringsfristen.

Spørsmål 1: Refleksjon

Dette er en refleksjonsoppgave. Denne oppgaven teller ikke mot antall oppgaver som må løses for å få godkjent øvingen. For mer informasjon om refleksjonsoppgavene, se nettsidene.

Refleksjonsoppgavene handler kun om refleksjon rundt forståelse av og arbeid med fagstoffet og øvingen. Pass på så du ikke inkluderer noen sensitive personopplysninger i refleksjonen din.

Før du begynner å gjøre øvingen, reflekter over følgende:

- Hvilke deler av pensum knyttet til denne øvingen er du usikker på?
- Hva tror du at du kommer til å lære ved å gjøre denne øvingen?

Etter du er ferdig med øvingen, reflekter over følgende:

- Hva forstår du nå, som du ikke forsto før du begynte å jobbe med øvingen?
- Hvilke nye sammenhenger ser du?
- Kan du komme på flere anvendelser av stoffet fra denne øvingen? I så fall hvilke?
- Er det noe du fortsatt er usikker på? I så fall hva?
- Hvis du skulle gjort øvingen på nytt, hva ville du gjort annerledes?

Jeg forstår veldig lite. Jeg lurer på hva jeg må gjøre for å forstå det bedre. Lese mer, gjøre flere eksempeloppgaver. Vet ikke.

Spørsmål 2: Sorteringsalgoritmer

Hvilken funksjon er byttet ut med UNKNOWN her?

```
QUICKSORT(A, p, r)
1  if p < r
2      q = UNKNOWN(A, p, r)
3      QUICKSORT(A, p, q - 1)
4      QUICKSORT(A, q + 1, r)
```

- ☒ PARTITION
- ☐ MERGE
- ☐ QUICKSORT
- ☐ INSERTION-SORT

Spørsmål 3: Sorteringsalgoritmer

Denne oppgaven handler om QUICKSORT og MERGE-SORT. Med «sorteringsarbeid» menes her den faktiske flyttingen av tall som en sorteringsalgoritme gjør. Hvilket av følgende alternativ er sant?

- ☐ MERGE-SORT gjør sorteringsarbeidet før det rekursive kallet, mens QUICKSORT gjør det etterpå.
- ☐ Begge algoritmene gjør sorteringsarbeidet etter det rekursive kallet.
- ☐ Begge algoritmene gjør sorteringsarbeidet før det rekursive kallet.
- ☒ QUICKSORT gjør sorteringsarbeidet før det rekursive kallet, mens MERGE-SORT gjør det etterpå.

Spørsmål 4: Sorteringsalgoritmer

Hvilke av utsagnene er sant om å kjøre PARTITION(A, 1, n) der n er lengden til A?

- ☒ Etter vi har kjørt PARTITION, er vi garantert at A[1] ≤ A[n].
- ☐ A vil forbli uforandret dersom A[n] er ekte større enn alle de andre elementene i A før vi kjører PARTITION.
- ☒ A vil være sortert etter å ha kjørt PARTITION uansett hvordan A ser ut før vi kjører metoden.
- ☐ Hvis A er sortert før vi kaller PARTITION, vil kjøretiden til funksjonskallet være Θ(n²).

Spørsmål 5: Sorteringsalgoritmer

La A = ⟨1, 8, 3, 4, 1, 2, 7, 6, 8⟩. Hva er A etter å ha kjørt MERGE(A, 3, 4, 7)?

- ☒ ⟨1, 8, 1, 2, 3, 4, 7, 6, 8⟩
- ☐ ⟨1, 8, 1, 2, 3, 4, 6, 7, 8⟩
- ☐ ⟨1, 1, 2, 3, 4, 6, 7, 8, 8⟩
- ☐ ⟨1, 8, 3, 1, 2, 4, 7, 6, 8⟩

Spørsmål 6: Sorteringsalgoritmer

La A være en tabell sortert i stigende rekkefølge, og B en tabell sortert i synkende rekkefølge. Hvilken påstand stemmer da om kjøretiden til QUICKSORT?

- ☐ Kjøretiden for A er Θ(n²), og for B er kjøretiden Θ(n lg n).
- ☐ Kjøretiden for B er Θ(n²), og for A er kjøretiden Θ(n lg n).
- ☐ Begge har kjøretid Θ(n lg n).
- ☒ Begge har kjøretid Θ(n²).

Spørsmål 7: Sorteringsalgoritmer

Hvilke alternativ under er korrekt?

(Det er finnes mange varianter av MERGE-SORT og QUICKSORT. Bruk beskrivelsen av algoritmene fra pensum/CLRS.)

- ☒ QUICKSORT sorterer på stedet (in place).
- ☐ MERGE-SORT kan få Θ(n²) kjøretid hvis tabellen allerede er sortert.
- ☐ MERGE-SORT sorterer på stedet (in place).
- ☐ QUICKSORT har en kjøretid på Θ(n lg n) i verste tilfelle.

Spørsmål 8: Sorteringsalgoritmer

Hvilke av følgende utsagn om **RANDOMIZED-QUICKSORT** stemmer?

- ☒ En sortert tabell vil ikke alltid ha kjøretid $\Theta(n^2)$.
- ☐ Splittelementet (*pivot*) velges alltid som miderste element for å få en jevn partisjon.
- ☐ **RANDOMIZED-QUICKSORT** har en kjøretid på $O(n \lg n)$ i verste tilfelle.
- ☐ Når vi har valgt splittelement (*pivot*), bytter det plass med det første elementet i tabellen.

Spørsmål 9: Merge sort

Her skal du implementere **MERGE** og **MERGE-SORT** som beskrevet i CLRS.

Den eneste forskjellen er at i din implementasjon er **A** 0-indeksert, siden **A** er en liste i Python.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester [her](#).

```
1 def merge(A, p, q, r):
2     # Skriv din kode her
3
4
5 def merge_sort(A, p, r):
6     # Skriv din kode her
```

Spørsmål 10: Substitusjonsmetoden

Du har fire algoritmer **A**₁, **A**₂, **A**₃ og **A**₄, som hver har kjøretid henholdsvis bestemt av funksjonene **T**₁(*n*), **T**₂(*n*), **T**₃(*n*) og **T**₄(*n*). Du ønsker å teste om kjøretiden til hver av de fire algoritmene er $\Omega(n^3)$ ved hjelp av substitusjonsmetoden. Du forutsetter for hver av dem at den induktive hypotesen $T_i(n) \geq c \cdot n^3$ stemmer. Etter å ha gjennomført substitusjonsmetoden for hver av funksjonene får du resultatene

$$T_1(n) \geq c \cdot n^3 - 5n^2$$

$$T_2(n) \geq c \cdot n^4 + 5n^2$$

$$T_3(n) \geq c \cdot n^3 + n$$

$$T_4(n) \geq c \cdot n^3 - 1$$

Hvilke av algoritmene har du greid å bevise at har kjøretid $\Omega(n^3)$? Anta at grunntilfellene i den matematiske induksjonen også stemmer.

- ☐ **A**₃
- ☒ **A**₄
- ☒ **A**₁
- ☐ **A**₂

Spørsmål 11: Unimodal tabell

Denne oppgaven er basert på en eksamensoppgave.

En *unimodal tabell* er en tabell med tall som, etter å ha blitt rotert et bestemt antall ganger, kan deles i to intervaller, der det første er strengt stigende, og det andre strengt synkende. Med rotasjon mener vi her å ta det første elementet ut av tabellen og sette det inn bak alle de andre elementene i tabellen.

Et eksempel er tabellen $\langle 8, 6, 3, 1, 5, 9, 12, 10 \rangle$. Hvis man roterer denne tabellen 3 ganger ender man opp med $\langle 1, 5, 9, 12, 10, 8, 6, 3 \rangle$, som har egenskapen at tabellen først er strengt stigende og deretter strengt synkende. $\langle 8, 6, 3, 1, 5, 9, 12, 10 \rangle$ er derfor en unimodal tabell.

Merk at en av eller begge intervallene kan være tomme. For eksempel vil tabellene $\langle 1, 2, 6, 10 \rangle$ og $\langle 10, 6, 2, 1 \rangle$, samt tomme tabeller, være unimodale.

I en slik unimodal tabell er det en bestemt største verdi. Du skal implementere en algoritme med kjøretid $\Theta(\lg n)$ som finner denne verdien.

Testene vil ikke godkjenne løsninger som prøver å skrive til tabellen, f.eks. `x[0]=1`, og du vil få en feilmelding.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester [her](#).

- Hint 1
- Hint 2
- Hint 3

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er basert på hvor mange ganger elementene i tabellen aksesseres.

```
1  # x er her en unimodal liste
2  def find_maximum(x):
3      # Skriv din kode her
4
5  # Sett 'highscore' til True hvis du vil vises på poengtavlen.
6  # For mer info se 'https://algdattidi.ntnu.no/ovinger.html#poengtavle'
7  # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
8  highscore = False
```

Spørsmål 12: Masterteoremet

La $T(n) = 32 \cdot T(n/4) + 3n^3 + 7n$ være en algoritmisk rekurrens. Hvilket tilfelle tilhører rekurrensen når du benytter masterteoremet?

- ☒ Tilfelle 3
- ☐ Tilfelle 1
- ☐ Ingen av dem
- ☐ Tilfelle 2

Spørsmål 13: Masterteoremet

La $T(n) = 64 \cdot T(n/4) + 3n^3 + 7n$ være en algoritmisk rekurrens. Hva blir $T(n)$?

- ☐ $\Theta(n^4)$
- ☒ $\Theta(n^3 \lg n)$
- ☐ $\Theta(n^3)$
- ☐ $\Theta(n^4 \lg n)$

Spørsmål 14: Masterteoremet

Denne oppgaven er basert på en eksamensoppgave.

La $T(n) = 3 \cdot T(n/9) + \Theta(\sqrt{n} \lg n)$ være en algoritmisk rekurrens. Hva blir $T(n)$?

- ☐ $\Theta(\lg^2 n)$
- ☐ $\Theta(\lg n)$
- ☐ $\Theta(n \lg^2 n)$
- ☐ $\Theta(\sqrt{n} \lg n)$
- ☐ $\Theta(n \lg n)$

☒ $\Theta(\sqrt{n} \lg^2 n)$

Spørsmål 15: Kjøretidsanalyse

Funksjonen `FUNCTION-C` under har kjøretid $\Theta(\sqrt{n})$. Hva blir kjøretiden til funksjonen `FUNCTION-A`, som funksjon av n ?

```
FUNCTION-A(n)
1  FUNCTION-C(n)
2  if n > 1
3      FUNCTION-A(n/3)
4      FUNCTION-B(n)

FUNCTION-B(n)
1  FUNCTION-C(n)
2  if n > 1
3      FUNCTION-A(n/3)
```

Hint 1

- ☐ $\Theta(n^2)$
- ☐ $\Theta(n \lg n)$
- ☒ $\Theta(n^{\log_3(2)})$
- ☐ $\Theta(n)$
- ☐ $\Theta(\lg n)$

Spørsmål 16: Kjøretidsanalyse

Hva blir kjøretiden til funksjonen `FUNCTION-D`, som funksjon av n ?

```
FUNCTION-D(n)
1  if n > 42
2      FUNCTION-D(n - 42)
3      FUNCTION-D(42)
```

- ☐ $\Theta(1)$
- ☐ Den vil aldri stoppe
- ☐ $\Theta(\lg n)$
- ☒ $\Theta(n)$
- ☐ $\Theta(n \lg n)$

Spørsmål 17: Kjøretidsanalyse

Hva blir kjøretiden til funksjonen `FUNCTION-E`, som funksjon av n ? Anta at `PRINT` tar konstant tid.

```
FUNCTION-E(n)
1  for i = 1 to n
2      for j = i + 1 to n
3          PRINT(“Algdat ruler!”)
4  if n > 1
5      for i = 1 to 36
6          FUNCTION-E(n/6)
```

- ☐ $\Theta(\lg n)$
- ☐ $\Theta(n \lg n)$
- ☐ $\Theta(n^3)$
- ☐ $\Theta(\sqrt[5]{n^6} \lg n)$
- ☐ $\Theta(n)$
- ☐ $\Theta(n^2)$
- ☒ $\Theta(n^2 \lg n)$
- ☐ $\Theta(1)$

Spørsmål 18: Kjøretidsanalyse

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

Funksjonen **FUNCTION-G** under utfører k^2 operasjoner når den kalles med argument k . Hva blir kjøretiden til funksjonen **FUNCTION-F**, som funksjon av n ?

```
FUNCTION-F(n)
1  FUNCTION-G(n/6)
2  if n > 1
3      FUNCTION-F(n/2)
4      FUNCTION-F(n/2 − 1)
5      FUNCTION-F(n/2 − 2)
6      FUNCTION-F(n/2 − 3)
```

Hint 1

- ☐ $\Theta(\lg n)$
- ☐ $\Theta(n^2)$
- ☐ $\Theta(n)$
- ☐ $\Theta(n \lg n)$
- ☐ $\Theta(n^2 \lg n)$

Spørsmål 19: Rekursjonstrær

La $T(n) = T(n/10) + T(n/5) + T(n/\pi) + n^3$ være en algoritmisk rekurrens, der $T(1) = 1$. Hva er høyden på rekursjonstreet?

- ☐ $\Omega(n^3)$
- ☐ $\Theta(n \lg n)$
- ☐ $\Theta(n)$
- ☐ $\Theta(n^2)$
- ☐ $\Theta(n^3)$

☐ $\Theta(\lg n)$

Spørsmål 20: Rekursjonstrær

La $T(n) = 4 \cdot T(\lfloor \sqrt{n} \rfloor) + n^3$ være en algoritmisk rekurrens, der $T(2) = 1$. Hva er høyden på rekursjonstreet?

Hint 1

- ☐ $\Theta(\lg n)$
- ☐ $\Theta(n)$
- ☐ $\Theta(\lg \lg n)$
- ☐ $\Theta(\sqrt{n})$

Spørsmål 21: Rekurrens med ulikheter

La $T(n)$ være en algoritmisk rekurrens, slik at $T(n) \geq 7 \cdot T(n/4) + \Theta(n \lg n)$. Hva stemmer om $T(n)$?

- ☐ $\Theta(n \lg n)$
- ☐ $O(n \lg n)$
- ☐ $\Omega(n \lg n)$
- ☐ $\Omega(n^{\log_4 7})$
- ☐ $\Theta(n^{\log_4 7})$
- ☐ $O(n^{\log_4 7})$

Spørsmål 22: Rekurrens med ulikheter

La $T(n)$ være en algoritmisk rekurrens, slik at $T(n) \leq 3 \cdot T(n/4) + \Theta(n \lg n)$ og $T(n) \geq 2 \cdot T(n/3) + O(\sqrt{n})$. Hva stemmer om $T(n)$?

- ☐ $O(n \lg n)$
- ☐ $\Omega(n \lg n)$
- ☐ $O(n^{\log_3 2})$
- ☐ $\Theta(n^{\log_3 2})$
- ☐ $\Omega(n^{\log_3 2})$
- ☐ $\Theta(n \lg n)$

Spørsmål 23: Største prisme

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

Du er i et basseng fylt med vann. Bassenget har en bunn som er flislagt med identiske kvadratiske fliser med sidelengde på 1 meter. Bassenget er helt kvadratisk, med n fliser i lengden og n fliser i bredden. Dette er likevel ikke et helt vanlig basseng, fordi flisene ikke nødvendigvis ligger i samme dybde under vannflaten.

Du ønsker å gjemme en kiste i bassenget, slik at hele kisten ligger under vannflaten (toppen kan være akkurat på vannflaten). Du har ikke laget kisten enda, men har bestemt at kisten skal ha form som et rett rektangulært prisme (et prisme der alle sider er rektangler). Kisten skal også ligge parallelt med flisene.

Det finnes flere slike basseng, og du ønsker å finne største mulige kiste, i volum, for hvert av bassengene. For å gjøre det enklere for deg selv, skal du ikke regne ut dette for hånd; du skal skrive en algoritme som finner dette volumet for deg.

Input til algoritmen er en todimensjonal liste med dybde under vannflaten til flisene i meter.

Algoritmen din må ha $O(n^4)$ som gjennomsnittlig kjøretid for å få godkjent.

Eksempel: Hvis listen du får inn er `[[1, 1], [2, 1]]`, vil dette tilsvare et basseng som er slik:

1 m	1 m
2 m	1 m

Her ser man at høyden på kisten kan være maksimalt 2 meter, siden det er det dypeste punktet i bassenget. Hvis kisten skal være 2 meter høy, kan den kun ligge på den flisen som er på 2 meter dybde, noe som gir et volum på $1\text{ m} \cdot 1\text{ m} \cdot 2\text{ m} = 2\text{ m}^3$.

Hvis vi derimot har en kiste som er 1 meter høy, kan den ligge oppå alle flisene, siden alle flisene ligger minst en meter under vannflaten. Da får vi en kiste med volum $2\text{ m} \cdot 2\text{ m} \cdot 1\text{ m} = 4\text{ m}^3$. Størrelsen på det største mulige volumet i dette eksempelet er dermed 4 m^3 .

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester [her](#).

Hint 1

Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er basert på hvor lang tid det tar å løse et sett med problemer.

Hint: Det finnes algoritmer med kjøretid på $O(n^4)$ i gjennomsnitt.

```
1  # Her er x en n*n-liste med dybden på flisene
2  # Funksjonen skal returnere volumet på den største
3  # mulige kisten (i kubikkmeter) som beskrevet over.
4  def largest_cuboid(x):
5      # Skriv koden din her
6
7  # Sett 'highscore' til True hvis du vil vises på poengtavlen.
8  # For mer info se 'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
9  # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
10 highscore = False
```

Spørsmål 24: Anbefalte ekstraoppgaver

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Her følger en liste over anbefalte ekstraoppgaver, som kan være nyttig å se på om du ønsker litt ekstra trening eller som repetisjon mot eksamen. Oppgavene er hentet fra [gamle eksamensoppgaver](#), *Introduction to Algorithms* av Cormen mfl. (CLRS) og [Kattis](#) og er sortert basert på kilde.

- [2026S](#) - 13, 14
- [2025H](#) - 14
- [2024S](#) - 14, 16, 20
- [2023H](#) - 3
- [2023S](#) - 7, 12
- [2022H](#) - 7, 16
- [2022S](#) - 7
- [2021H](#) - 5, 7
- [2021S](#) - 6
- [2020H](#) - 6

- [2019H](#) - 11, 12
- [2018H](#) - 11, 16, 17
- [2018S](#) - 14, 15, 16
- [2017H](#) - 2c
- [2017S](#) - 14
- [2016H](#) - 10, 11
- [2016S](#) - 6, 7, 11
- [2015H](#) - 16, 17
- [2015S](#) - 2, 3, 13
- **CLRS** - 2.3-4
- **CLRS** - 4.3 - alle (de første er lettere enn de siste, så velg utfra vanskelighetsgrad)
- **CLRS** - 4.4 - alle (de første er lettere enn de siste, så velg utfra vanskelighetsgrad)
- **CLRS** - 4.5-1, 4.5-3, 4.5-5
- **CLRS** - 4-3, 4-5, 4-6
- **CLRS** - 7.2-2, 7.2-5
- **CLRS** - 7.3-1
- **CLRS** - 7-2, 7-5, 7-7
- **Kattis** - <https://open.kattis.com/problems/guess> (lett)
- **Kattis** - <https://open.kattis.com/problems/froshweek> (medium)
- **Kattis** - <https://open.kattis.com/problems/factorfree> (vanskelig)

Send inn